1. (3 Punkte) a. Benenne im folgenden Code alle Beispiele für die folgenden Begriffe: globale Variable, lokale Variable, Parameter und Argument.
   b. Was wird ausgegeben?

```
k = 10
def doit(y):
    r = 5
    w = k + y + r
    return w

z = doit(3) + k
print(z)
```

Lösung:

a. k, z sind globale Variablen, y, r, w sind lokale Variablen, 3 ist Argument, y ist Parameter.
b. 28

2. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
returns: string, die Zeichenfolge 'x.'

>>> func2()
'x.'
'''
```

Lösung:

```
def func2():
    return 'x.'
```

3. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
returns: None,  Gibt die Zeichenfolge 'x.' aus

>>> func1()
x.
'''
```

Lösung:

```
def func1():
    print('x.')
```

4. (3 Punkte) Gib an, was ausgegeben wird. Tritt ein Fehler auf, gib an, in welcher Zeile und warum.

```
def f(x):
    print(x + 1)

def g(x):
    return x + 1

a = f(3)              # Zeile 7
b = g(3)              # Zeile 8
print(a, b)           # Zeile 9
print(g(g(1)))        # Zeile 10
f(g(1))               # Zeile 11
print(f(1) + 1)       # Zeile 12
```

Lösung:

```
4           Zeile 7: f gibt 4 aus, a bekommt None
None 4      Zeile 9
3           Zeile 10: g(1) = 2, g(2) = 3
3           Zeile 11: g(1) = 2, f gibt 3 aus
2           Zeile 12: f(1) gibt 2 aus und liefert None,
            danach Fehler (TypeError): None + 1 ist nicht möglich
```

5. (2 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
x: nicht leerer string
returns: String, der nur aus dem ersten und letzten Zeichen von x besteht.

>>> teil('Hallo')
'Ho'

>>> teil('H')
'HH'
'''
```

Lösung:

```
def teil(x):
    return x[0]+x[-1]
```

6. (2 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
s string, k >= 0, k optional mit default 1
returns: string, die ersten k zeichen von s

>>> make('Hallo')
'H'
>>> make('Hallo',3)
'Hal'
'''
```

Lösung:

```
def make(s, k=1):
    return s[:k]
```

7. (3 Punkte) Gegeben ist die folgende Funktion:

```
def preis(netto, mwst=19, rabatt=0):
    return netto * (100 + mwst) // 100 - rabatt
```

Gib an, was ausgegeben wird, bzw. ob ein Fehler auftritt.

```
print(preis(100))
print(preis(100, 7))
print(preis(100, rabatt=10))
print(preis(rabatt=5, netto=200))
print(preis(50, 7, 3))
print(preis(mwst=7))
```

Lösung:

```
119     mwst und rabatt haben ihre Standardwerte
107     mwst = 7
109     mwst = 19, rabatt = 10
233     Schlüsselwortargumente, Reihenfolge egal: 238 - 5
50      53 - 3
Fehler (TypeError): für netto gibt es keinen Standardwert
```

8. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
x: int
returns 'A' falls x < 20
        'B' falls 20 <= x < 30
        'C' falls 30 <= x

>>> func(15)
'A'
>>> func(25)
'B'
>>> func(35)
'C'
'''
```

Lösung:

```
def func(x):
    if x < 20: return 'A'
    if 20 <= x < 30: return 'B'
    return 'C'
```

9. (3 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben. Die Funktion soll mit `return` verlassen werden, sobald das Ergebnis feststeht.

```
'''
a: Liste mit Zahlen
returns: True, wenn die Liste aufsteigend sortiert ist, sonst False

>>> ist_sortiert([1, 3, 3, 7])
True
>>> ist_sortiert([1, 5, 2, 7])
False
>>> ist_sortiert([])
True
'''
```

Lösung:

```
def ist_sortiert(a):
    for i in range(len(a) - 1):
        if a[i] > a[i + 1]:
            return False        # erstes Paar in falscher Reihenfolge
    return True
```

10. (1 Punkt) Was wird ausgegeben?

```
x = 10
def doit(k):
    global x
    x = 5
    z = 7
    w = k + z
    return w

z = doit(3) + x
print(z)
```

Lösung: 15

11. (4 Punkte) Was erscheint auf der Konsole?

```
def a(x,y):
    return x + 2 * y
def b(x):
    return x - 3
def c(x,y):
    return (x + 3) // y
def d(x,y):
    return x > b(y)
z = c(a(3,1),b(7))
print(d(z,5),z)
```

Lösung: False 2

12. (4 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
n: int, n > 0
returns: die Summe der echten Teiler von n. Echte Teiler sind alle Teiler der Zahl,
         außer die Zahl selbst. Beispiel: die Summe der echten Teiler
         von 12 ist: 1+2+3+4+6 = 16

>>> echteTeilerSumme(12)
16
'''
```

Lösung:

```
def echteTeilerSumme(n):
    summe = 0
    for k in range(1,n):
        if n % k == 0:
            summe +=k
    return summe
```

13. (3 Punkte) Eine natürliche Zahl heißt *vollkommen*, wenn sie gleich der Summe ihrer echten Teiler ist, z.B. 6 = 1+2+3. Verwende die Funktion `echteTeilerSumme` aus der vorigen Aufgabe.

    (a) Schreibe eine Funktion `ist_vollkommen(n)`, die `True` zurückgibt, wenn `n` vollkommen ist, sonst `False`.

    (b) Schreibe eine Funktion `vollkommene_zahlen(bis)`, die eine Liste aller vollkommenen Zahlen von 1 bis `bis` zurückgibt. `vollkommene_zahlen(500)` liefert `[6, 28, 496]`.

Lösung:

```
def ist_vollkommen(n):
    return echteTeilerSumme(n) == n

def vollkommene_zahlen(bis):
    ergebnis = []
    for k in range(1, bis + 1):
        if ist_vollkommen(k):
            ergebnis.append(k)
    return ergebnis
```

14. (4 Punkte) Gegeben sind die Signaturen einiger Java-Methoden:

```
static int maximum(int a, int b)
static void begruessen(String name)
static boolean istGerade(int n)
static double kreisflaeche(double r)
```

(a) Gib für jede Methode an, welche Parameter (mit Datentyp) sie hat und ob und welchen Wert sie zurückgibt.

(b) Schreibe `maximum` und `begruessen` als Python-Funktionen mit Typhinweisen. `begruessen('Ida')` soll `Hallo Ida!` ausgeben.

Lösung:

Methode	Parameter	Rückgabewert
`maximum`	`a`, `b` vom Typ `int`	ein `int`
`begruessen`	`name` vom Typ `String`	keiner (`void`)
`istGerade`	`n` vom Typ `int`	ein `boolean`
`kreisflaeche`	`r` vom Typ `double`	ein `double`

```
def maximum(a: int, b: int) -> int:
    if a > b:
        return a
    return b

def begruessen(name: str):
    print(f'Hallo {name}!')
```